

TRACE: Structural Planning, Tool-Grounded Multi-Agent Verification, and Traceability-Guided Local Repair for LLM-Based Java Unit Testing

Nguyen Tuan Dung¹, Vuong Kieu Anh¹, Doan Huu Minh Quang¹, Ngo Duc Chinh¹, Tran Van Han¹

¹FPT University, Hanoi, Vietnam.

Contributing authors: dungnthel180616@fpt.edu.vn; anhvkhe180546@fpt.edu.vn;
quangdhmhe181540@fpt.edu.vn; chinhndhe180889@fpt.edu.vn;
hantvhe181281@fpt.edu.vn;

Abstract

Large language model (LLM)-based Java unit-test generation remains limited by weak structural planning, uncontrolled repair, and unreliable verification. TRACE combines auditable scenario planning, traceability-guided local repair, and tool-grounded multi-agent verification. We evaluated these components in three studies and compared the integrated pipeline with EvoSuite. In the planning study, 600 runs on 150 SF110 methods and two model configurations showed that planning increased aggregate execution success from 58.00% to 88.33%, while using approximately twice as many tokens. Structural metrics improved after Holm correction, but differing non-missing denominators limit the branch-coverage comparison. In the repair study, scenario-local repair succeeded on 60% of 100 defect units, versus 46% for method-level repair and 76% for full-class regeneration; its advantage over method-level repair was not significant after correction. In the exploratory verification study, Gemini false acceptance was 26.92% for the specialized verifier versus 57.69% for a general reviewer, and repair-routing accuracy was 66.67% versus 42.59%, at higher inference cost. Across 450 paired runs, TRACE’s integrated pipeline delivered verified suites in 55.11% of cases versus 28.89% for EvoSuite ($p = 0.0001$). This comparison used a TRACE-specific verification gate whose neutrality across generators remains unestablished. The results show improved delivered reliability in the evaluated settings, alongside cost and validity limits.

Keywords: unit test generation; large language models; structural planning; traceability-guided repair; multi-agent verification; Java

1 Introduction

Software testing is an essential but labor-intensive activity in the software development lifecycle because it directly affects the quality, reliability, and maintainability of the system. Among these, unit testing provides the first line of verification by checking small program units such as classes and methods. Detecting errors at this

level helps shorten the process of pinpointing causes, limits errors from spreading to integration and deployment stages, and supports refactoring and regression detection. However, manually building a high-quality unit test suite—including input data, edge cases, error paths, and appropriate assertions—still requires significant time and knowledge of both the source code and the project environment [6, 26, 32].

The development of large language models (LLMs) opens up the possibility of automating this activity. Thanks to their interpretation and code generation capabilities, LLMs can analyze the focal method, propose test scenarios, generate JUnit code, and construct assertions from the provided context. Recent studies show that LLMs can generate tests achieving significant coverage [6, 26, 28, 32]. Adaptive focal context along with dependency and build information at the project level helps increase the practical validity of the output [5, 6, 26], while industrial deployment emphasizes the role of pre-execution filters before accepting improved tests [3]. However, a useful test suite not only needs to compile and run but also must achieve the intended structural objectives, detect faults, and maintain quality; therefore, line/branch coverage [15], mutation score [23], and test-smell evidence [22] provide additional perspectives rather than mutually substitutive measures.

Existing methods still face a significant gap between generating test code and producing a usable test suite. LLMs may call non-existent APIs, choose incorrect imports or dependencies, violate type or access rules, or create fixtures and mocks incompatible with the class to be tested. Many tests thus fail to compile; tests that compile successfully can still fail at runtime due to incorrect initialization, unexpected exceptions, or assertions that do not match observed behavior. ChatTester, ChatUniTest, TestART, and Test-Loter use feedback from compilation, execution, or coverage in generation-validation-repair cycles [6, 11, 31, 32]; CATGen emphasizes structured project context, deterministic scaffolding, and static post-processing [5]; while large-scale studies show that results are still sensitive to prompt and context configuration [25, 26]. These advances do not automatically ensure that tests have correctly implemented the target scenario, have a reliable oracle, or have been fixed within a scope that does not break already verified tests.

The literature reviewed here leaves three specific gaps. First, standardized test design techniques such as equivalence partitioning, boundary-value analysis, and structural coverage provide auditable rules to infer test conditions [14], while

prior LLM-based work has explored method slicing and testing-intention prompts [20, 28]. Therefore, the performance of a package that adds structural planning before generation remains unclear when the LLM, context, test-method cap, and generation budget are fixed, but the added planning call and its compute are retained. Second, ChatTester, ChatUniTest, TestART, and Test-Loter use feedback to fix test failures [6, 11, 31, 32]; among the approaches reviewed here, we did not identify published documentation of an immutable bidirectional contract between the atomic scenario and the test method. The benefits of scenario-local repair in terms of cost, edit scope, repair iterations, and regression therefore need to be measured directly against fixing a failing test method without scenario context and regenerating the entire class. Third, MAGISTER separates the Analyzer, Test Generation, Executor, and Refiner Agent, while CANDOR uses multi-agent coordination for oracle generation [1, 29]. Among the approaches reviewed in this study, we did not identify a framework that jointly separates provenance-linked verification evidence streams, preserves immutable scenario identity, and aggregates them into a primary failure cause and repair route at the scenario level. The performance of this specialized-verification package needs to be distinguished from Basic Tool Gates and a General-Purpose Reviewer provided with the same evidence.

TRACE addresses these gaps through a seven-block pipeline. (1) Input and Repository Analysis receives a Java repository, focal class, and focal method, then extracts build metadata, dependencies, and AST/CFG into a repository context. (2) Structural Planning applies predicate-induced partitions, boundary values, branch outcomes, and exception paths to create an Immutable Atomic Scenario Plan consisting of $S_0, S_1, \dots, S_N$. (3) Traceable Scenario Test Generation maintains a bijective mapping $S_i \leftrightarrow T_i$ between each scenario and exactly one test method. (4) Tool Execution uses Maven/Gradle and JUnit to produce build/run evidence, employs JaCoCo and PIT for coverage/mutation measurements linked to individually executed tests, and uses tsDetect to assess test smells at the test file or generated class level [15, 17, 22, 23]. (5) Specialized Multi-Agent Verification for

the Conformance Agent, Compilation/Runtime Diagnosis Agent, and Coverage/Mutation Analysis Agent handles independent aspects once the corresponding evidence is ready, then the Evidence Aggregator merges the results according to the scenario ID. (6) Repair Routing distinguishes scenario-local defects, shared-infrastructure defects, and context-insufficient cases; local repair only changes test methods under ownership and must go through a regression check. (7) The Final Test Suite stores test code, scenario-test mapping, verification evidence, repair history, and unresolved/abstained cases.

This study has three main contributions. First, Auditable Test-Design-Technique-Grounded Structural Planning: the study proposes a Structural Planner separated from the Generator, combining repository context with AST/CFG to infer predicate-induced partitions, boundary values, branch outcomes, and exception paths according to standardized test design techniques [14]. The Planner releases an immutable atomic scenario plan in which each scenario has an ID, activation conditions, expected behavior with source evidence, and a specific structural target at the line, branch/predicate outcome, or exception path level. Relative to method-slicing and testing-intention prompts [20, 28], this representation allows tracing each scenario back to the structure and oracle evidence that generated it, auditing target completion level and clarifying testing objectives before code generation.

Second, Bijective Scenario Ownership and Regression-Checked Scenario-Local Repair: the study establishes a bijective mapping between atomic scenarios and test methods, so that each S_i owns the correct T_i and each T_i realizes exactly one S_i ; verification evidence and repair history keep the scenario ID intact throughout the pipeline. When a scenario-local defect is detected, TRACE only fixes or regenerates the owned test method, then reruns the target test and previously verified tests before accepting the fix. This mechanism differs from repair rounds that do not declare immutable scenario-test ownership [6, 11, 31, 32]: it limits edit authority to the method level and turns the regression on the verified test into a measurable acceptance gate, rather than assuming that the local repair always reduces the Changed LOC or regression relative

to every baseline. Third, Tool-Grounded Specialized Multi-Agent Verification for Scenario-Level Failure Attribution: the study proposes specialized verification agents using evidence generated by Maven/Gradle and JUnit for build/run [17], JaCoCo for coverage [15], and PIT for mutation analysis [23]; tsDetect can provide auxiliary test-smell evidence at the file/class level [22], but test-smell outcomes are not evaluated in the present studies and no empirical claim rests on that signal. The Conformance Agent, Compilation/Runtime Diagnosis Agent, and Coverage/Mutation Analysis Agent handle eligible tasks according to their respective evidence streams; the Evidence Aggregator links reports with scenario IDs, identifies false acceptance, attributes the primary failure cause, and selects scenario-local, shared-infrastructure, or context-insufficient routes. Unlike the role-specialized generation/refinement pipeline of MAGISTER [1], TRACE keeps the provenance of each verification report separately and aggregates them using a decision schema at the scenario level. Tool outputs can be reproduced under a fixed repository state, tool version, command, and configuration; the agent’s decision is still considered an inference that needs to be evaluated in RQ3, and is not described as deterministic.

Finally, the empirical evaluation part of the study is designed to directly verify the three contributions above rather than just describing the architecture. RQ1 compares Structural Planning with Direct Generation under the conditions of keeping the LLM, repository context, test-method cap, and generation budget fixed; RQ2 evaluates Traceability-Guided Scenario-Local Repair against failing-test-method repair without scenario context and full-class regeneration under the same repair budget; RQ3 measures the accuracy of Tool-Grounded Specialized Multi-Agent Verification compared to basic tool gates and a general-purpose reviewer provided with the same tool evidence. The results are reported according to compilation/execution success, structural-target satisfaction, branch coverage, mutation score, token/API cost, edit scope, repair iterations, regression on previously verified tests, false acceptance, scenario-level failure attribution, and primary repair-route accuracy.

2 Related Work

2.1 Unit Test Generation with LLMs and Structural Planning

Automated unit test generation has evolved from search-based and random-generation techniques such as EvoSuite [8] and Randoop [21] to LLM-based approaches, although industrial evaluations indicate that the assertion quality of the previous tools is still limited [2]. Early LLM methods used the focal method as the main context: ChatUniTest [6] applies a Generation–Validation–Repair process, while TestPilot [26] uses re-prompting based on execution errors, though it has only been evaluated on JavaScript/npm. Subsequent studies expanded the context and testing intentions: HITS [28] breaks the focal method into code slices, Test Intention [20] incorporates testing intention into the prompt, and CATGen [5] incorporates project dependencies and structured context. Overall, these studies show the value of selecting context and testing intentions, but the literature reviewed here does not establish the individual impact of a structured planning phase separate from code generation under equivalent controlled conditions.

2.2 Test Traceability and Automated Repair

TRACETS4J [16] builds test–focal-method links at the data level across large Java project sets. Compilation/execution feedback has supported test repair in ChatUniTest [6], TestPilot [26], and TestART [11]. However, the scope of modifications and the impact of each change on previously verified tests still need to be controlled, especially when multiple tests share fixtures or dependencies. tsDetect [22] adds maintenance signals through test smell detection but does not replace evidence of assertion correctness.

2.3 Tool-Grounded Verification and Agent Architectures

SymPrompt [25] and CoverUp [24] use execution paths and coverage feedback to guide test generation, although high coverage does not guarantee strong assertions. Mutation testing provides this additional signal: MUTGEN [27], ACH [7], AdverTest [4] improve the ability to kill mutants,

while TOGILL [13] evaluates oracles in terms of correctness and fault detection strength—showing that coverage, mutation, and oracle quality are signals that cannot completely replace each other. At the actor architecture level, MetaGPT [12] and SWE-agent [30] establish role-playing patterns and tool interactions; SGAgent [33] combines a localizer, suggester, and fixer with a repository-level knowledge graph. In test generation, Retry and Refine [10] use seven agents but only evaluate on MBPP/HumanEval; CANDOR [29] focuses on improving the oracle through multi-agent coordination; MAGISTER [1] separates analyzer, generator, executor, and refiner roles. These works demonstrate the potential of coordinating roles and tool feedback, but the literature reviewed here does not isolate the value of a specialized verifier given the same evidence compared with a simpler verification mechanism. A surviving mutant, for example, does not automatically prove the oracle is weak if the relationship with the focal method and the corresponding test objective has not been established.

2.4 Synthesis and research gaps

From the research directions above, three prominent gaps are: (1) the independent impact of a structured planning phase separated from code generation, under the condition of keeping the generation model and budget fixed; (2) how to maintain traceability at the scenario level to limit the scope of bug fixes and restrict regression on verified tests; and (3) how to organize tool evidence according to clearly defined acceptance conditions, combined with specialized verification roles for each type of evidence. This study considers all three gaps within an auditable generate–verify–fix lifecycle, presented in detail in Section 3.

3 Methodology

This study proposes TRACE, an end-to-end framework used to generate, verify, debug, and package Java unit tests at the focal method level. TRACE uses a Large Language Model (LLM), but it does not entrust the entire process to a single LLM call. The approach is designed according to three requirements stated in the Introduction: the test scenario must be planned from auditable test design techniques; each generated test method

must have clear ownership of the scenario; and verification decisions must be based on execution evidence from the tool. As illustrated in Figure 1, the sequential pipeline maintains traceability from the repository to the final test suite, while dedicated verification agents can run in parallel on independent evidence streams.

The framework takes a Java repository, a focal class, a focal method, and a pipeline configuration file. The output includes accepted test code, scenario-test mapping, verification evidence, repair history, and unresolved or abstained cases. This design serves three research questions (Research Question-RQ): RQ1 separates the impact of Structural Planning from Direct Generation; RQ2 evaluates Traceability-Guided Scenario-Local Repair against Failing-Test-Method Repair and Full-Class Regeneration; RQ3 assesses Tool-Grounded Specialized Multi-Agent Verification compared to Basic Tool Gates and a General-Purpose Reviewer when methods receive the same tool evidence.

3.1 Input analysis and repository

The first module builds a repository context to bind all decisions in the subsequent steps. TRACE identifies the Java version, build tool, module path, test framework, dependencies, source-test directory structure, Application Programming Interface (API) of the focal class, signature of the focal method, existing requirement/oracle sources, and related tests. Integrating the build, dependencies, and focal context into the pipeline is directly compatible with ChatUniTest and CATGen [5, 6], while also being consistent with project-level experimental settings in recent studies [25, 26].

The repository analysis function is represented as follows:

$$M = f_{\text{analyze}}(R, c, m, Y) \quad (1)$$

In which: R is the raw Java project repository, including source code, test code, build files, and the resources necessary for analyzing or executing the project.

c is the focal class selected in R ; this is the class containing the behavior that the study wants to test. m is the focal method belonging to c ; this is the target method that the Structural Planner analyzes and the generated tests are aimed at.

Y is the pipeline.yaml configuration file, containing input parameters, build environment, LLM, planning, generation, verification, repair, metric, output, and scenario selection policy. f_{analyze} is the repository analysis and normalization function. It combines data extracted from R , c , and m with the configuration in Y to create a unified input representation for the entire pipeline. M is the normalized repository context; M is the output of f_{analyze} and is the common metadata source for the Planner, Generator, tool layer, verification agents, and repair layer. In this study, the structure of M is determined as follows:

$$M = (\text{java_version}, \text{build_tool}, \text{dependencies}, \text{test_framework}, \text{source_paths}, \text{test_paths}, \text{focal_class_api}, \text{focal_method_signature}, \text{complexity_map}, \text{existing_tests}, \text{requirement_evidence}, \text{build_commands}, \text{ast_model}, \text{cfg_model}, \text{input_config}, \text{repo_config}, \text{build_config}, \text{llm_config}, \text{prompt_config}, \text{planning_config}, \text{generation_config}, \text{verification_config}, \text{repair_config}, \text{metrics_config}, \text{output_config}, \text{report_config}) \quad (2)$$

The groups `java_version`, `build.tool`, `dependencies`, `test.framework`, `source.paths`, `test.paths`, `focal.class.api`, `focal.method.signature`, `complexity.map`, `existing.tests`, `requirement.evidence`, and `build.commands` are extracted or inferred from R , c , and m . Among them, `complexity.map` stores structural features needed for selecting test targets; `focal.class.api` describes constructors, fields, and methods that generated tests can use; `requirement.evidence` stores contracts/Javadoc, behaviors confirmed by existing tests, invariants observable in the repository, and the corresponding provenance; `build.commands` are normalized Maven or Gradle commands for the target repository. When requirement/oracle sources in the repository are contradictory or insufficient, M records an indeterminate state instead of creating expected behavior without basis.

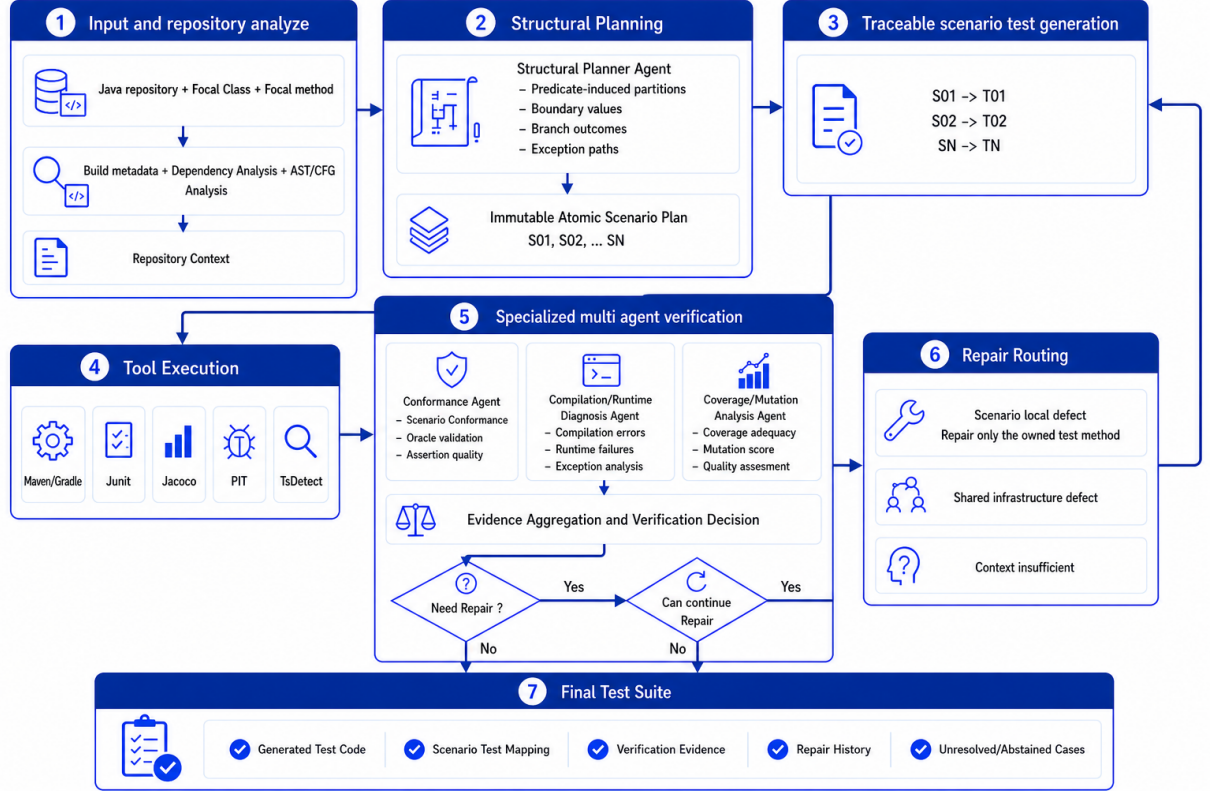


Fig. 1 TRACE workflow from repository analysis and immutable scenario planning to traceable test generation, tool execution, and specialized verification. Repair routing feeds eligible defects back to generation; the final artifact records accepted tests, mappings, evidence, repair history, and unresolved or abstained cases

`ast_model` is an Abstract Syntax Tree (AST) model, representing the tree form of a program’s syntactic structure. `cfg_model` is a Control-Flow Graph (CFG) model, in which nodes represent statements or basic blocks and edges represent possible control transfers. From the AST and CFG, TRACE collects predicates, predicate outcomes, branches, exceptional exits, and simple data dependencies of `m` to support Structural Planning. The `input_config`, `repo_config`, and `build_config` groups describe the input, repository location, and build environment. `llm_config` and `prompt_config` describe the model, inference settings, and prompt template. `planning_config`, `generation_config`, `verification_config`, and `repair_config` define the budget and policy for the four corresponding stages; `planning_config` also contains the test-method cap and scenario-selection policy that were fixed before execution. `metrics_config` selects measures for coverage,

mutation, and test smell; `output_config` and `report_config` define the artifacts and reports to be saved. These fields come from `Y` and do not change the meaning of the structured data extracted from `R`.

The output of the module is a repository-context object with structure, not a free-form prompt. As a result, the Planner uses structured facts; the Generator uses valid APIs, imports, and test frameworks; the tool layer uses the correct build command and path; and the repair layer reuses the same metadata to limit the scope of modifications.

3.2 Auditable Structural Planning

The second module realizes the contribution C1—Auditable Test-Design-Technique-Grounded Structural Planning. Before generating test code, the Structural Planner Agent creates an immutable atomic scenario plan based on

equivalence partitioning, boundary-value analysis, predicate-induced partition, branch outcome, and exception/error path. A predicate is a Boolean condition that governs control flow; a predicate-induced partition is the input domain divided according to the outcomes of that condition. These techniques are consistent with the role of test-design techniques in the International Organization for Standardization/International Electrotechnical Commission/Institute of Electrical and Electronics Engineers (ISO/IEC/IEEE) 29119-4:2021 standard [14].

The planning function is represented as follows:

$$\Pi = P_{\text{struct}}(M, c, m) = \{S_i \mid 1 \leq i \leq n\} \quad (3)$$

Here, P_{struct} is the Structural Planning function; the function uses the repository context M and the focal target pair (c, m) to infer the plan Π . The symbol Π denotes the set of immutable atomic scenarios; n is the total number of scenarios in the plan; i is the index of the scenario, running from 1 to n ; and S_i is the i -th scenario. M , c , and m retain the meanings defined in Section 3.1. Each atomic scenario S_i is represented as follows:

$$S_i = (\text{id}_i, \text{technique}_i, \text{trigger}_i, \text{input_partition}_i, \text{expected_behavior}_i, \text{oracle_provenance}_i, \text{structural_target}_i, \text{required_context}_i) \quad (4)$$

In the above tuple, **id.i** is the stable identifier of the scenario; **technique.i** records the test design technique that generated the scenario; **trigger.i** is the activation condition; **input.partition.i** is the input class or boundary region from which values need to be taken; **expected.behavior.i** is the observable behavior that the oracle must check; **oracle.provenance.i** points to the requirement, contract/Javadoc, already verified test, or invariant serving as the basis for the expected behavior; **structural.target.i** is the line, branch, predicate outcome, or exception path that the scenario targets; **required.context.i** lists the fixture, object state, dependency, or exception condition needed to execute the scenario. A scenario without

sufficiently reliable oracle provenance is marked as uncertain and must not be automatically accepted into the test suite.

Plan Π remains unchanged after release. If a step fails due to test implementation, TRACE fixes the implementation test method T_i rather than silently changing the scenario set. If evidence shows that the expected behavior is unfounded, oracle sources are conflicting, or the structural target is infeasible, the scenario is moved to the abstained/unresolved state with provenance instead of being corrected like a test defect. Therefore, the plan can be audited by counting and comparing equivalence classes, boundary values, branch outcomes, and exception paths before the test code is accepted. In contrast to methods that use code slices or testing-intention prompts [20, 28], the planning unit of TRACE is the structural target with oracle provenance and can be compared with Direct Generation under the same generation budget in RQ1.

3.3 Traceable Scenario-Based Test Generation

The third module implements the first part of the contribution C2-Bijective Scenario Ownership and Regression-Checked Scenario-Local Repair. For each S_i in Π , TRACE generates exactly one test method T_i . This relationship prevents the Generator from mixing multiple independent scenarios into the same method or creating methods without a responsible scenario.

The test function according to the scenario is represented as follows:

$$T_i = G_{\text{trace}}(S_i, M, \theta), \quad 1 \leq i \leq n \quad (5)$$

In which, G_{trace} is the traceable scenario test generation function; S_i is the source scenario; M is the repository context; θ is the set of inference parameters of the LLM, such as model identifier, temperature, context limit, and generation limit; T_i is the generated test method implementing S_i . The index i and the limit n retain the meaning from Section 3.2.

Candidate test suite and scenario-test mapping are defined as follows:

$$\begin{aligned} T &= \{T_i \mid 1 \leq i \leq n\}, \\ \Omega &= \{(S_i, T_i) \mid 1 \leq i \leq n\} \end{aligned} \quad (6)$$

In which, T is the set of candidate test methods and Ω (Omega) is the set of scenario–test mapping pairs. Ω is bijective within the generated suite: each S_i maps to exactly one T_i and each T_i maps back to exactly one S_i ; therefore $|\Omega| = |T| = |S| = n$.

Ownership invariant is recorded as follows:

$$\text{owner}(T_i) = S_i \wedge \text{implements}(T_i) = S_i \quad (7)$$

In this, `owner` is a function that returns the scenario owning T_i ; `implements` is a function that returns the scenario that T_i implements; and the symbol $\wedge$ means that both conditions must be true simultaneously. T_i stores `id.i` in a machine-readable form, for example in the method name, annotation, structured comment, or sidecar mapping. The prompt generating T_i only contains edit permissions for the method owned by S_i . Ω therefore provides an attribution and repair unit smaller than the whole-file level [6, 11, 31, 32].

3.4 Tool Execution and Evidence Collection

The fourth module executes generated tests in the target repository and collects tool-grounded evidence. TRACE calls Maven or Gradle according to M , then runs tests using a framework compatible with the project, such as JUnit. If compilation is successful, the system collects execution results, stack traces, and assertion outcomes. For a valid run, TRACE collects branch/line coverage using Java Code Coverage (JaCoCo) and mutation evidence using PIT. tsDetect is run on the generated test file/class to collect test-smell evidence at the level reported by the detector. These tools support multi-dimensional evaluation rather than relying only on compilation pass/fail [15, 17, 22, 23].

To attribute to each scenario, TRACE collects `compile`, `run`, `assertion outcome`, `coverage`, and `mutation` evidence for each pair (S_i, T_i) by running T_i separately or using an equivalent test filtering mechanism; then the framework reruns the related test set to check interactions and regression. Since the default output of tsDetect is aggregated by test file, smell findings are stored as

class/suite-level quality evidence and are not used alone to assign the primary failure cause for S_i . If an analysis implementation temporarily contains only T_i along with the required scaffolding, the report must clearly specify the isolation procedure and provenance before the smell is linked to the scenario. When a measurement is unavailable due to build or tool limitations, the unavailable status is recorded instead of being inferred by an LLM.

The evidence record of the pair (S_i, T_i) is represented as follows:

$$\begin{aligned} E_i &= (\text{compile}_i, \text{run}_i, \\ &\quad \text{assertion_outcome}_i, \text{coverage}_i, \\ &\quad \text{mutation}_i, \text{logs}_i, \text{provenance}_i) \end{aligned} \quad (8)$$

In which, E_i only contains the observable evidence for scenario i : `compile.i` stores the compilation state and compiler diagnostics; `run.i` stores the execution state and runtime failure; `assertion_outcome.i` stores assertion pass/fail observed by the test runner but does not itself conclude whether the oracle is correct; `coverage.i` records the achieved line/branch targets; `mutation.i` records related mutants that were killed, survived, or not yet evaluated; `logs.i` stores raw compiler messages, stack traces, and tool output; `provenance.i` stores tool version, command, configuration, repository state, and report path. Scenario conformance, oracle validity, and assertion quality are not included in E_i but are inferred by the Conformance Agent in the next module. The tsDetect test-smell report is stored separately at the class/suite scope as described above.

Tool execution is separated from LLM reasoning. With repository state, tool version, command, and fixed configuration, the tool provides measurements that can be reproduced under controlled conditions; agents in the subsequent module explain the evidence but are not allowed to replace missing evidence with speculation. Builds/tests that show flaky behavior or are environment-dependent are marked as uncertain and are not called deterministic. This distinction also does not assume that the behavior of the LLM is deterministic.

3.5 Specialized Multi-Agent Verification

The fifth module implements the C3 contribution—Tool-Grounded Specialized Multi-Agent Verification for Scenario-Level Failure Attribution. The Conformance Agent checks the relationship between T_i and S_i , oracle provenance, and assertion quality without assuming that these conclusions are already present in the raw evidence. Compilation/Runtime Diagnosis Agents analyze compiler diagnostics, missing imports, dependency conflicts, runtime exceptions, fixture construction errors, and assertion failures. Coverage/Mutation Analysis Agents check structural-target satisfaction, branch contribution, and mutation evidence. Agents run in parallel when the corresponding evidence in E_i is ready; agents without evidence are required to return unavailable/abstain instead of speculating.

The three agents generate the reports $V_{\text{conf},i}$, $V_{\text{run},i}$, and $V_{\text{covmut},i}$. $V_{\text{conf},i}$ is the report on scenario conformance, oracle validity, and assertion quality; $V_{\text{run},i}$ is the compilation/runtime diagnosis report; $V_{\text{covmut},i}$ is the coverage/mutation adequacy report. The reports retain their own evidence provenance so that the Evidence Aggregator can verify the agent’s conclusions against the raw tool output. The Evidence Aggregator aggregates them using the following function:

$$D_i = f_{\text{aggregate}}(S_i, T_i, E_i, V_{\text{conf},i}, V_{\text{run},i}, V_{\text{covmut},i}) \quad (9)$$

In this, $f_{\text{aggregate}}$ is the aggregation function according to a fixed decision schema; the function reconciles scenario, test, raw tool evidence, and three specialized reports. D_i is the verification decision for scenario i , including acceptance status, primary failure cause, evidence provenance, confidence or uncertainty level, and proposed repair route. The schema prioritizes mandatory build/run gates; when these gates are achieved, the system then considers conformance/oracle and structural adequacy. If the reports conflict or required evidence is missing, the system does not select accept but moves to an appropriate indeterminate state. The symbols S_i , T_i , and E_i retain the meanings defined in previous sections.

The route space includes accept, `repair_scenario_local`,

`repair_shared_infrastructure`, and `abstain_context_insufficient`. Accept is only chosen when the required gates, scenario conformance, and oracle validity are all satisfied; `repair_scenario_local` is chosen when the error is due to the implementation of T_i ; `repair_shared_infrastructure` is chosen when the error lies in a fixture, helper, dependency, or shared build configuration; `abstain_context_insufficient` is chosen when the evidence is insufficient, the oracle is conflicting, or the scenario/target cannot be safely verified. Therefore, a test that compiles successfully can still be rejected if S_i is not implemented, the structural target is not touched, or the expected behavior is not checked. MAGISTER separates the roles of analyzer/generator/executor/refiner [1]; TRACE instead maintains independent specialized verification reports, attaches provenance to immutable scenario IDs, and synthesizes primary failure attribution along with the repair route. In RQ3, this design is compared with Basic Tool Gates and a General-Purpose Reviewer receiving the same tool evidence.

3.6 Repair Routing and Scenario-Local Repair

The sixth module completes the C2 contribution by routing repair according to D_i . With scenario-local defect, TRACE only fixes or regenerates T_i owned by S_i . Repair input includes S_i , T_i , M , E_i , D_i , and the relevant part of the tool log; the repair prompt does not allow rewriting test methods that are not owned and have been verified.

The local repair function is represented as follows:

$$T'_i = f_{\text{repair_local}}(S_i, T_i, M, E_i, D_i, b_i) \quad (10)$$

In which, $f_{\text{repair_local}}$ is the function to repair or regenerate a single method T_i based on the scenario, repository context, evidence, and diagnosis; b_i is the remaining repair budget of scenario i ; T'_i is the repaired version of T_i , and the prime symbol $'$ indicates a new candidate after a repair iteration. S_i , M , E_i , and D_i retain the same definitions as in the previous sections. After the repair,

TRACE compiles and reruns T'_i , collects the target evidence, and then runs a regression check on previously verified tests that may be affected by class-level setup or shared fixtures. T'_i only replaces T_i when the local fault is fixed and no previously verified test changes from pass to fail. The edit scope, number of iterations, token/API cost, repair success, and regression are recorded to answer RQ2.

With shared-infrastructure defects, TRACE switches to the shared repair procedure and reruns the entire affected scenario set; the defect is not incorrectly assigned to an individual S_i . For context-insufficient, oracle-conflict, or invalid-scenario cases, the system abstains and records the case as unresolved along with a reason code and provenance instead of continuing to ask the LLM to guess repository information or modify the test to match an unreliable scenario. Plan Π is not changed in the current run; any future plan edits must create a new version and cannot be considered scenario-local repair. Each round stops when a candidate is accepted, b_i is exhausted, or the route requests abstention.

3.7 Final Test Suite and linkage with evaluation

The seventh module packages the final output according to the expression:

$$A = (T_{\text{final}}, \Omega_{\text{final}}, E_{\text{final}}, H_{\text{repair}}, U_{\text{abstain}}) \quad (11)$$

In which, A is the final artifact package; T_{final} is the set of accepted test methods; Ω_{final} is the scenario-test mapping that is still valid; E_{final} is the final set of verification evidence; H_{repair} is the repair history, including edit scope, iteration, and regression results; U_{abstain} is the set of unresolved or abstained scenarios along with the reason and evidence provenance. No unaccepted tests are included in T_{final} .

This output is directly linked to the experimental design. RQ1 compares Structural Planning with Direct Generation under the same LLM, repository context M , generated-method cap, and generation budget by compilation/execution success, structural-target satisfaction, focal-method branch coverage, and mutation score. If $-\Pi$ exceeds the cap, the `planning.config` in Y

applies the scenario-selection policy that was fixed before running; both selected and omitted scenario IDs are recorded, and the Generators of both treatments receive the same method cap and generation budget. Planning cost is recorded separately so as not to be mixed with the generation budget. RQ2 compares Traceability-Guided Scenario-Local Repair with Failing-Test-Method Repair and Full-Class Regeneration from the same initial failing artifacts, E_i , LLM, and repair budget measured by token/API cost, edit scope, repair iteration, regression, repair success, and final test adequacy. RQ3 compares $f_{\text{aggregate}}$ and specialized agents with Basic Tool Gates and General-Purpose Reviewer using false acceptance, scenario-level failure attribution, and primary repair-route accuracy.

By releasing an auditable plan Π , maintaining the bi-directional Ω , storing raw evidence E_i separately from agent inference, and only accepting repairs after regression checks, TRACE transforms LLM-based unit-test generation into a traceable workflow. The framework does not consider compilation success or the convincing commentary of the LLM as sufficient; the planned scenario, oracle provenance, generated method, tool evidence, verification decision, and regression result must be consistent before a test is included in T_{final} . Test-smell evidence from tsDetect is kept within the scope that the detector actually reports and is not inferred into scenario-level attribution if isolation provenance is missing.

4 Experiments and Results

This section presents the experimental design used to evaluate the three main contributions of the proposed method. Three research questions are identified as follows:

RQ1 – The effectiveness of Structural Planning.

Under the conditions of using the same LLM, repository context, limits on the number of generated test methods, and generation budget, how does Structural Planning based on test-design techniques affect compilation/execution success, structural-target satisfaction, focal-method branch coverage, and mutation score compared to Direct Generation without planning?

RQ2 – Effectiveness of Traceability-Guided Scenario-Local Repair.

With the same initial failing scenario–test artifacts, tool evidence, LLM, and repair budget, does Traceability-Guided Scenario-Local Repair help reduce token/API cost, edit scope, number of repair iterations, and regressions on previously verified tests, while still maintaining repair success and final test adequacy, compared to Failing-Test-Method Repair without scenario context and Full-Class Regeneration?

RQ3 – Accuracy of Tool-Grounded Specialized Multi-Agent Verification.

Compared to Basic Tool Gates and a General-Purpose Reviewer provided with tool evidence, how accurately can Tool-Grounded Specialized Multi-Agent Verification detect false acceptance, perform scenario-level failure attribution, and select the primary repair route?

RQ1 evaluates the impact of the structured planning step before test code generation. RQ2 evaluates the ability to use scenario–test traceability relationships to limit the scope of repair and protect the tests that have been verified. RQ3 evaluates the multi-agent verification mechanism in synthesizing tool evidence, detecting invalid artifacts, and correctly identifying both the cause and the repair direction.

A separate system-level comparison with EvoSuite evaluates the integrated TRACE pipeline rather than another individual component.

4.1 Experimental design

Three component studies addressed RQ1–RQ3 using separate datasets, and an additional integrated evaluation compared the complete TRACE pipeline with EvoSuite. Each run was performed on an independent workspace at a fixed repository revision. The experimental protocol called for preserving the build configuration, prompt, model, toolchain, and output artifacts; these empirical artifacts are not included in this manuscript repository. Maven/Gradle, JUnit, JaCoCo, and PIT respectively provided evidence for compilation, execution, coverage, and mutation testing [15, 17, 23].

For RQ1, DIRECT and PLANNED were compared using a paired design on the same 150 focal methods from 58 Java projects in SF110 [9]. DIRECT generates tests directly from the

repository context, whereas PLANNED creates atomic scenarios from AST/CFG before generating test code. In each model configuration, both treatments used the same focal methods, source revision, repository context, decoding settings, a maximum limit of six generated test methods, and generation budget; repair and verifier feedback were disabled. Structural-Target Satisfaction of both was calculated on the same reference target set, determined independently before test generation. The experiment used Gemini 3.7 Flash and DeepSeek V4 Flash; each focal method–model–treatment combination was run once, resulting in a total of 600 runs (150 focal methods $\times$ two models $\times$ two treatments). Planning costs are recorded separately and included in the total costs of PLANNED. Because PLANNED adds a planning call and was not matched to DIRECT on total calls or total tokens, this is a package-level comparison of planning plus its additional inference, not a causal isolation of the plan representation under equal compute.

For RQ2, TGSLR, FTMR, and FCR were evaluated on the same 100 initial failing artifacts across six LIBRO–Defects4J projects [18, 19]. The treatments used the same repository state, failure evidence, model, seed, repair budget, and stopping conditions. TGSLR received scenario–test mapping and only fixed the corresponding test method; FTMR fixed the same method but without scenario context; FCR was allowed to regenerate the entire test class. Each treatment ran once on each unit using the Gemini 3.6 Flash model, generating a total of 300 experimental runs, with up to three repair rounds and a nominal budget of 75,000 tokens per run. The budget is checked between completed calls, so the last call may cause the total tokens to exceed the threshold; provider retries are not counted as a new experimental unit.

For RQ3, B0, B1, and B2 were evaluated on a separate verification dataset consisting of 73 cases from 14 Java projects, including 65 controlled-injection cases and eight natural cases. B0 only used compilation and selected-JUnit outcomes; B1 and B2 received the same evidence envelope, in which B2 used three specialized verifiers and an Evidence Aggregator. Providing the same evidence does not imply equal reasoning budgets; the number of calls and token counts were recorded to assess the cost trade-offs. The author team reports

that it settled the final gold labels through oral discussion; no case-level record of that review was kept. The cohort had been exposed to a previous prototype, and the Qwen sensitivity run was selected after inspecting completed alternatives; RQ3 is therefore exploratory.

The integrated TRACE-EvoSuite evaluation used 150 frozen SF110 targets with three fixed repetitions per target, yielding 450 paired observations under a common evaluator and treatment-blinded B2 gate. It measured delivered-system outcomes rather than isolating the causal effect of any one component; its design and results are reported in Section 4.6.

4.2 Measurement and analysis methods

RQ1 uses Compilation Success Rate, Execution Success Rate, Structural-Target Satisfaction, focal-method Branch Coverage, and Mutation Score. In the end-to-end analysis, tests that do not compile or do not execute receive a value of 0 for structural metrics. A supplementary analysis on the pairs that both execute successfully is used to evaluate conditional quality. Cost is measured by token usage and wall-clock time.

RQ2 uses core repair success, token/API cost, number of repair rounds, Changed LOC, and regression-producing repair attempts. A core repair is considered successful if it is applied, compiles, makes the focal test pass on the fixed revision, exhibits bug-revealing behavior on the buggy revision if needed, and does not leave any regression. Coverage and mutation are only reported descriptively on successful repairs with measurement evidence. Since the data does not yet have independent scenario conformance and structural-target satisfaction, RQ2 does not evaluate the entire acceptance contract of the framework. RQ3 uses decision accuracy, False-Acceptance Rate (false acceptances divided by the 26 false-passing cases), False-pass F1 with false-pass detection as the positive class, attribution Macro-F1 across cause labels, repair-route accuracy on the 54 REPAIR_REQUIRED cases, decision coverage over all 73 cases, and selective accuracy among covered decisions. Provider-invalid or partially invalid outputs are retained through their case-level aggregate decisions and are also counted as output

errors. ABSTAIN is a valid label for three-class decision accuracy and can be correct when the gold label is also ABSTAIN; it is excluded from decision coverage and selective accuracy, and it is neither an acceptance nor a repair prediction in the false-passing analysis. Reported abstention rates use all 73 cases as their denominator. Cost is measured by token usage and number of agent calls.

The McNemar exact test is used for binary outcomes and the Wilcoxon signed-rank test for paired continuous metrics where specified. RQ1 applies Holm correction within each model’s five-metric primary family and reports matched odds ratios or paired rank-biserial correlations; its conditional analysis is explicitly unadjusted. RQ2 applies Holm correction to a ten-test family in an exploratory reanalysis of the 100-unit cohort and reports the stated effect sizes and unadjusted paired-bootstrap intervals. The underlying RQ2 campaign reports retain provisional power and outcome plans, so these tests are not presented as preregistered confirmatory evidence. RQ3 is also exploratory: its McNemar tests and bootstrap intervals are unadjusted unless a table states otherwise. The integrated comparison uses the Holm family specified in its protocol and a hierarchical bootstrap. Most component configurations have one run per unit-treatment; the integrated comparison instead has three repetitions per target. In all studies, clustering and the recorded analysis contract limit the scope of inference.

4.3 RQ1 Results: The Impact of Structural Planning

This section presents the results of RQ1 to evaluate the impact of Structural Planning on the reliability, test adequacy, and cost of the unit test generation process. Two approaches were compared: DIRECT, in which the LLM generates tests directly from the repository context, and PLANNED, in which the LLM constructs a structural planning consisting of atomic scenarios before generating test code. The experiments used Gemini 3.7 Flash and DeepSeek V4 Flash on the same 150 focal methods from 58 projects for each model-approach combination, totaling 600 generation runs. Within each model configuration, both approaches used the same focal methods, source revision, repository context, decoding settings, a maximum limit of 6 generated test methods, and

generation budget. Adaptive Repair was turned off so that the results directly reflect the quality of the initial generation.

According to the end-to-end scoring protocol, a run with an invalid test artifact, or one that does not compile or execute successfully, receives a value of 0 for Structural-Target Satisfaction (STS), Branch Coverage (BC), and Mutation Score (MS). This calculation simultaneously reflects the reliability of the test-generation pipeline and the adequacy of the generated test suite. N/A is only used when a metric is undefined due to the absence of a structural target or a valid denominator; these values are excluded from the calculation of the corresponding metric. DIRECT and PLANNED are paired by focal method, and statistical analysis is performed separately for each LLM.

4.3.1 Descriptive Results

Table 1 summarizes the end-to-end results for both treatments and model configurations.

CSR: Compilation Success Rate; ESR: Execution Success Rate; STS: Structural-Target Satisfaction; BC: focal-method Branch Coverage; MS: focal-method Mutation Score. CSR and ESR have $N = 150$ for each model-approach combination. STS, BC, and MS are averages over non-N/A observations. For Gemini, the number of STS/BC/MS observations is 140/139/150 for both approaches; for DeepSeek, the corresponding numbers are 148/148/150 in DIRECT and 141/140/150 in PLANNED. The available aggregate does not include case-level N/A reason codes, and the DeepSeek STS/BC denominators differ by treatment. Consequently, the STS/BC means—including the aggregate BC values highlighted below—are not common-cohort contrasts and may reflect differential missingness; they remain descriptive until reason-coded missingness, common-pair, and sensitivity analyses are available. The summary rows only provide aggregate descriptions and should not be used to infer or rank the two LLMs. Across both model configurations, PLANNED achieved a CSR of 96.33%, 17.33 percentage points higher than DIRECT’s 79.00%. ESR increased from 58.00% to 88.33%, corresponding to 91 additional generated test suites executing successfully. For structural-adequacy metrics, STS increased descriptively

from 35.52% to 58.47%, BC from 43.51% to 70.58%, and MS from 41.43% to 63.92%. These aggregated results show a consistent trend, but are only descriptive.

On Gemini 3.7 Flash, PLANNED improved all five metrics: CSR increased by 3.33 percentage points, ESR by 7.33 percentage points, STS by 11.84 percentage points, BC by 11.85 percentage points, and MS by 9.41 percentage points. The increases on DeepSeek V4 Flash were larger, at 31.34, 53.34, 33.48, 41.34, and 35.58 percentage points, respectively. Overall, Structural Planning is associated with more generated test suites passing the compilation/execution gates and satisfying more structural targets.

4.3.2 Paired Statistical Analysis

Because DIRECT and PLANNED are evaluated on the same focal methods, statistical analysis follows a paired design. The two-sided exact McNemar test is used for CSR and ESR. The two-sided Wilcoxon signed-rank test is used for STS, BC, and MS on the PLANNED - DIRECT differences, with tie correction, zero differences removed, and no continuity correction applied. Differences are formed at the decimal precision of the exported values so that floating-point subtraction does not split tied ranks. Holm correction is applied for the family of five primary metrics within each LLM, with $\alpha = 0.05$. Matched odds ratio (OR) is reported for binary metrics, while paired rank-biserial correlation (r_{rb}) is used for continuous metrics.

Table 2 reports the paired tests and Holm-adjusted results.

n is the number of complete pairs before removing zero differences in the Wilcoxon test. CSR, ESR, and MS each have 150 pairs in each LLM; STS/BC have 140/139 pairs on Gemini and 141/140 pairs on DeepSeek. The numbers in parentheses after OR are respectively the number of pairs where PLANNED succeeded-DIRECT failed and the number of pairs in the opposite direction. $OR = \infty$ indicates that there are no pairs in the opposite direction in the sample, which does not mean infinite effectiveness. $r_{rb} < 0$ favors PLANNED; $r_{rb} = 1$ indicates that all non-zero paired differences are positive.

These complete-pair tests already compare the same focal methods within each metric. They do

Table 1 End-to-end descriptive results

Model	Treatment	CSR, n (%)	ESR, n (%)	STS (%)	BC (%)	MS (%)
Gemini 3.7 Flash	DIRECT	142 (94.67%)	124 (82.67%)	46.54	59.34	54.91
Gemini 3.7 Flash	PLANNED	147 (98.00%)	135 (90.00%)	58.38	71.19	64.32
DeepSeek V4 Flash	DIRECT	95 (63.33%)	50 (33.33%)	25.09	28.64	27.94
DeepSeek V4 Flash	PLANNED	142 (94.67%)	130 (86.67%)	58.57	69.98	63.52
Aggregate	DIRECT	237 (79.00%)	174 (58.00%)	35.52	43.51	41.43
Aggregate	PLANNED	289 (96.33%)	265 (88.33%)	58.47	70.58	63.92

STS, BC, and MS are end-to-end values: invalid, non-compiling, or non-executing runs receive 0; N/A is reserved for an absent structural target or valid denominator and is excluded from the corresponding mean.

Table 2 Results of paired tests

Model	Metric	n	p	p_{Holm}	Effect size
Gemini 3.7	CSR	150	0.0625	0.0625	OR = ∞ (5/0)
Gemini 3.7	ESR	150	0.000977	0.001953	OR = ∞ (11/0)
Gemini 3.7	STS	140	7.04×10^{-6}	2.11×10^{-5}	$r_{\text{rb}} = 0.907$
Gemini 3.7	BC	139	2.42×10^{-7}	1.21×10^{-6}	$r_{\text{rb}} = 1.000$
Gemini 3.7	MS	150	7.88×10^{-7}	3.15×10^{-6}	$r_{\text{rb}} = 1.000$
DeepSeek V4	CSR	150	5.52×10^{-12}	1.10×10^{-11}	OR = 16.667 (50/3)
DeepSeek V4	ESR	150	3.43×10^{-23}	1.72×10^{-22}	OR = 81.000 (81/1)
DeepSeek V4	STS	141	1.44×10^{-10}	1.44×10^{-10}	$r_{\text{rb}} = 0.906$
DeepSeek V4	BC	140	1.92×10^{-13}	5.76×10^{-13}	$r_{\text{rb}} = 0.953$
DeepSeek V4	MS	150	1.03×10^{-14}	4.13×10^{-14}	$r_{\text{rb}} = 0.984$

not make the non-N/A means in Table 1 common-cohort estimates or establish that missingness is independent of treatment and outcome. Reason-coded missingness and sensitivity analyses remain necessary before extending the complete-pair findings to the entire scheduled cohort.

After Holm correction, Gemini 3.7 Flash achieved statistically significant improvements in ESR, STS, BC, and MS; CSR did not reach the significance threshold ($p_{\text{Holm}} = 0.0625$). With DeepSeek V4 Flash, all five metrics reached statistical significance. The effect sizes for STS, BC, and MS were 0.907; 1.000; 1.000 on Gemini and 0.906; 0.953; 0.984 on DeepSeek, indicating that the paired differences mostly favored PLANNED. However, these tests used the focal method as the unit of analysis and have not adjusted for project clustering.

4.3.3 Conditional Quality Analysis

End-to-end scoring combines pipeline reliability with the adequacy of the generated test suite. Since PLANNED has a higher ESR, the increase in STS, BC, and MS may partly come from the reduction of unsuccessful runs scored 0. Therefore, a secondary analysis was conducted on the

focal methods that both DIRECT and PLANNED successfully executed. This is an exploratory available-pair analysis; the p-values have not been subjected to Holm correction and do not replace the primary end-to-end analysis. On Gemini, PLANNED increased STS by 9.70 percentage points ($n = 114$, $p = 0.000208$, $r_{\text{rb}} = 0.863$), BC by 7.83 percentage points ($n = 113$, $p = 1.78 \times 10^{-5}$, $r_{\text{rb}} = 1.000$), and MS by 6.25 percentage points ($n = 124$, $p = 5.91 \times 10^{-5}$, $r_{\text{rb}} = 1.000$).

On DeepSeek, the paired differences were smaller: STS increased by 1.18 percentage points ($n = 47$, $p = 0.528$, $r_{\text{rb}} = 0.286$), BC increased by 1.92 percentage points ($n = 47$, $p = 0.293$, $r_{\text{rb}} = 0.476$), and MS increased by 2.57 percentage points ($n = 49$, $p = 0.0431$, $r_{\text{rb}} = 1.000$). Only MS reached the 0.05 threshold before multiple-testing correction; this result would no longer reach the threshold if applying Holm correction for the three conditional metrics. The analysis by paired-executed subjects therefore does not support the previous assertion that conditional MS of PLANNED is lower: the comparison of 83.82% versus 73.29% was based on two sets of successful runs with different sizes and not a paired comparison. Evidence of executable-suite adequacy is

generally clear on Gemini but remains limited on DeepSeek.

4.3.4 Costs and Trade-offs (Cost Trade-off)

Structural Planning uses an additional LLM phase to create a plan, which increases costs. On Gemini, the mean wall-clock time increased from 13.45 s to 18.95 s (+40.92%), while mean token usage increased from 3,750 to 7,450 tokens per run; total tokens increased from 562,500 to 1,117,500 (+98.67%). On DeepSeek, the mean wall-clock time increased from 64.34 s to 100.06 s (+55.54%), while mean token usage increased from 7,904.10 to 16,189.38 tokens; total tokens increased from 1,185,615 to 2,428,407 (+104.82%). Meanwhile, the average number of generated test methods decreased from 3.65 to 2.92 on Gemini and from 5.74 to 2.92 on DeepSeek. Thus, the end-to-end improvement does not come from generating more test methods, but it comes with nearly double the token consumption and higher wall-clock time.

4.3.5 Answer to RQ1

Among the two evaluated model configurations, PLANNED statistically significantly improves ESR, STS, BC, and MS after Holm correction; CSR only reaches statistical significance on DeepSeek. Primary results indicate that the PLANNED package is associated with improved pipeline reliability and end-to-end test adequacy. Secondary paired-executed analysis shows that adequacy benefits are still clear on Gemini, but weak and unstable on DeepSeek. Therefore, the data does not yet support a generalized conclusion that PLANNED always produces stronger executable test suites than DIRECT across all LLMs. The achieved benefits also come with nearly double the token usage and higher wall-clock time. Because PLANNED includes an additional planning call and greater total inference compute, this package-level ablation does not causally isolate the structural-plan representation from the extra call or tokens. Each configuration only has one run per focal method, the available aggregate lacks N/A reason codes, and the tests have not yet accounted for project clustering or between-run variability. Therefore, the conclusion is limited to the dataset, model configurations, and budget that were examined.

4.4 RQ2 Results: Effectiveness of Traceability-Guided Scenario-Local Repair

4.4.1 RQ2 Experimental Setup

RQ2 compares Traceability-Guided Scenario-Local Repair (TGSLR), Failing-Test-Method Repair without scenario context (FTMR), and Full-Class Regeneration (FCR) on the same original faulty unit and tool outputs. The treatments use the same repository, model, budget, seed, and stopping conditions. TGSLR receives failure evidence along with scenario-test traceability and only repairs the test method that owns the scenario; FTMR receives the same failure evidence but without scenario/traceability; FCR receives scenario/traceability context and is allowed to regenerate the test class. Thus, the FCR contrast also changes edit scope; it is not a comparison with a context-free full-class generator.

The dataset consists of 100 distinct fault units from six LIBRO-Defects4J projects and 300 logic runs using models/gemini-3.6-flash. Each treatment is run once on each unit with seed 20260814, a 75,000-token budget, and a maximum of three repair rounds. Since the budget is checked between completed calls, the last call may exceed the limit; provider retries do not create new experimental units.

The analysis combines 297 runs from the 99-unit post-development batch with three runs for the replacement task D4J-JSOUP-32-48. The replacement was added after the original task D4J-COMPRESS-6-4 failed readiness checks. The source reports describe provisional power and outcome plans and eligibility-filtered comparisons; the present 100-unit analysis is therefore an exploratory reanalysis, not a retrospectively pre-registered confirmatory study.

All 100 matched treatment blocks (three treatments per fault unit) were retained; all outcomes other than Success—including two Cancelled runs of D4J-JACKSONXML-4-25 and an FTMR failure categorized as Unexpected for D4J-JACKSONCORE-17-10—were failures. Each pairwise contrast therefore contained 100 paired observations. Exact McNemar was used for success and regression; paired Wilcoxon was used for tokens, API call counts, and the number of revision rounds. Holm adjusted ten tests

from two comparisons $\times$ five outcomes. The analysis reported matched odds ratio or paired rank-biserial correlation along with unadjusted 95% paired bootstrap CI (10,000 resamples; seed 20260814). The number of changed lines and adequacy were only described because different subsets for success or measurements according to treatment were used. The adjusted tests and unadjusted intervals answer different multiplicity questions and should not be treated as interchangeable significance criteria.

4.4.2 Core Repair Success and Paired Comparison

The Success status in CSV requires the candidate to be applicable, compiled, and pass the focal test successfully on the fixed revision, the bug can be reproduced on the buggy revision when necessary, and no regression is caused. Since CSV does not have a scenario conformance or structural target field, this outcome only measures core repair, not the entire acceptance contract of TRACE; coverage and mutation are reported separately.

Table 3 reports core repair success for each treatment.

TGSLR succeeded in 60/100 runs, compared with 46/100 for FTMR and 76/100 for FCR. The schema-11 CSV exports contain explicit stage evidence rather than compilation information only in the stop reason. Their recorded compilation-stage statuses are Succeeded/Failed/NotRun = 72/15/13 for TGSLR, 79/11/10 for FTMR, and 86/0/14 for FCR. These describe the stage evidence retained at the end of each run, not whether any earlier candidate compiled; NotRun is not a compiler failure. Table 4 gives the paired core-repair comparisons.

For TGSLR-FTMR, $W/T/L = 21/72/7$, matched odds ratio = 3.00 (unadjusted 95% CI: 1.23–8.35) and $p_{\text{Holm}} = 0.0752$; the 14-point advantage does not reach the adjusted significance threshold. This is not evidence of equivalence. For TGSLR-FCR, $W/T/L = 4/76/20$, matched odds ratio = 0.20 (unadjusted 95% CI: 0.05–0.60) and $p_{\text{Holm}} = 0.0108$, favoring FCR for core repair success within this exploratory reanalysis.

4.4.3 LLM costs, number of revisions, and scope of edits

Table 5 summarizes inference cost, repair iterations, and edit scope.

Compared with FTMR, TGSLR used 2,953.3 fewer tokens, 0.31 fewer API calls, and 0.25 fewer repair rounds per run on average. None of these differences reached significance after Holm correction: $p_{\text{Holm}} = 0.1830, 0.7089$, and 0.0912 . Defining all cost differences as TGSLR minus comparator, the corresponding rank-biserial correlations are 0.230, -0.152, and -0.397; the unadjusted 95% mean-difference intervals are [-6,125.0; -96.7], [-0.98; 0.39], and [-0.46; -0.04]. A positive rank-biserial correlation denotes a tendency toward higher TGSLR cost; its sign need not match the arithmetic mean difference because ranks and means weight observations differently. Compared with FCR, TGSLR used 7,486.7 more tokens, 0.77 more API calls, and 0.66 more repair rounds; the corresponding p_{Holm} values were 3.82×10^{-6} , 0.00185, and 5.49×10^{-7} , with rank-biserial correlations of 0.585, 0.470, and 0.754 and unadjusted 95% intervals of [4,974.2; 10,153.9], [0.26; 1.27], and [0.46; 0.86]. Among successful repairs, TGSLR changed an average of 18.45 lines, compared with 16.50 for FTMR and 28.93 for FCR. These edit-scope differences are descriptive because the successful subsets differ.

The exports also record duration as the difference between run completion and start timestamps. Mean recorded durations over all 100 runs per treatment are 161.57 s for TGSLR, 187.23 s for FTMR, and 144.92 s for FCR. These are descriptive whole-run durations, including the work performed within each run, rather than isolated LLM latency or repair-only time. They are not additional outcomes in the ten-test Holm family and do not establish a statistically significant timing advantage.

4.4.4 Regression and final test adequacy

Table 6 reports regression events and the available adequacy measurements.

TGSLR and FTMR both recorded six regression-producing runs, so 94% of runs recorded no regression, compared with 92% for

Table 3 Core repair success per 100 defect units of each treatment.

Repair strategy	Runs	Successful core repairs	Core repair success rate
TGSLR	100	60	60.0%
FTMR	100	46	46.0%
FCR	100	76	76.0%

Table 4 Comparison of paired core repair success.

Comparison	Pairs	TGSLR success	Comparator success	Difference	TGSLR win/tie/loss
TGSLR-FTMR	100	60.0%	46.0%	+14.0 percentage points	21/72/7
TGSLR-FCR	100	60.0%	76.0%	-16.0 percentage points	4/76/20

FCR; both pairwise comparisons were not significant after correction ($p_{\text{Holm}} = 1.000$). These run-level percentages are not percentages of individual protected tests. For the available successful repair measures, the average line coverage and mutation score of TGSLR were higher than FTMR but lower than FCR. The outcomes ToolFailed, ReportMissing, and NotRun were considered as missing rather than 0, so adequacy was only interpreted descriptively.

4.4.5 Answering RQ2

The results do not establish overall superiority of TGSLR. Compared with FTMR, TGSLR achieved higher core repair success and lower mean token/API/iteration cost, but none of these paired differences remained significant after Holm correction; successful TGSLR patches also changed 1.95 more lines on average. Compared with FCR, successful TGSLR patches changed 10.48 fewer lines and TGSLR recorded two fewer regression events, but the exploratory tests favored FCR for success and token/API/iteration cost; the regression difference was not significant. These findings suggest a descriptive locality trade-off. Traceability is an architectural property of the repair contract, not a separately measured effectiveness gain. Missing coverage and mutation measurements on different successful subsets do not establish non-inferiority in final test adequacy. The evidence comes from one run per treatment, one model, six clustered projects, and an exploratory reanalysis of campaigns with provisional plans. Although the exports contain compilation-stage evidence and

whole-run duration, they do not independently measure scenario conformance, structural-target satisfaction, or oracle strength and therefore do not validate the full TRACE acceptance contract.

4.5 RQ3 Results: Evaluation of Tool-Grounded Specialized Multi-Agent Verification

RQ3. Compared with the Basic Tool Gate (B0) and a General-Purpose Reviewer using the same tool evidence (B1), how accurately does the Specialized Multi-Agent Verifier (B2) control false acceptance, attribute causes, and select the primary repair route?

4.5.1 Design and scope of reasoning

This is an exploratory v2 evaluation of 73 cases across 14 Java projects, including 65 controlled-injection cases and 8 natural cases. The author team reports that it reviewed and settled the final gold labels through oral discussion. No contemporaneous case-level record of individual judgments, disagreements, or final adjudication was kept. The accessible artifact contains the frozen case-level gold categories, decisions, causes, and acceptable routes. The number of independent human raters, their agreement, and the precise review procedure cannot be independently verified from these files. Earlier AI annotation passes and scripts in the workspace were tests, not evidence of the final gold-labeling procedure. Gold decision, cause, and repair route are mapped from the frozen category and tool/injection evidence; the validity of

Table 5 Costs, number of repair cycles, and number of lines changed.

Repair strategy	Mean tokens per run	Mean API calls per run	Mean repair iterations per run	Mean changed LOC per successful repair
TGSLR	25,284.9	4.51	2.05	18.4
FTMR	28,238.2	4.82	2.30	16.5
FCR	17,798.2	3.74	1.39	28.9

Table 6 Regression over the entire run and adequacy on successful repairs with measurements.

Repair strategy	Runs with regression	Coverage measurements / successful repairs	Mean line coverage	Mutation measurements / successful repairs	Mean mutation score
TGSLR	6	51/60	34.4%	40/60	17.6%
FTMR	6	39/46	31.3%	29/46	14.6%
FCR	8	59/76	37.9%	47/76	18.7%

the underlying categories cannot be independently audited against a human review record.

The dataset starts from developer-written tests in pinned public Java repositories. Each public case records the repository URL, revision, test ID, and source artifact hash. Eight included cases retain natural tests; the other 65 are controlled challenges constructed by intentionally changing test source, the stated scenario, or the available evidence to create a specified failure or context-insufficient condition. Thus, a controlled case does not always involve a source-code edit. The corresponding case and tool evidence are retained separately from private gold. The presence of a developer test in a public repository does not by itself establish the semantic correctness of every test or gold label.

The 73-case cohort was drawn from an earlier candidate pool of 180 and had been exposed to a previous prototype. There is no independently labeled, prospectively frozen holdout. The model comparisons below use the same frozen cohort and are interpreted as exploratory sensitivity analyses rather than independent validation.

B0 uses compilation and selected-JUnit results. B1 is a general reviewer. B2 consists of three specialists in plan-to-test conformance, compilation/runtime, and coverage/mutation; a deterministic aggregator creates the decision, primary cause, and repair route. The aggregator first selects the highest-priority valid DETECTED finding in the fixed order COMPILATION_ERROR, SHARED_INFRASTRUCTURE_ERROR, SETUP_OR_RUNTIME_ERROR, TARGET_NOT_REACHED,

WEAK_ORACLE, PLAN_TEST_MISMATCH, and ASSERTION_ERROR. If no such finding exists, it emits VERIFIED only when all three specialists return CLEAR with execution status OK; otherwise, including a partially invalid specialist set, it emits ABSTAIN with route REQUEST_TOOL_EVIDENCE. In each case, B1 and B2 receive an evidence envelope with the same SHA-256. The cohort has 26 false-passing cases. Its decision labels comprise 54 REPAIR_REQUIRED, 8 VERIFIED, and 11 ABSTAIN cases.

For metric calculation, let $N = 73$, let $f_i = 1$ denote a gold false-passing case, and let $\hat{f}_i = 1$ denote a prediction of REPAIR_REQUIRED with cause PLAN_TEST_MISMATCH, TARGET_NOT_REACHED, ASSERTION_ERROR, or WEAK_ORACLE. The false-passing confusion counts are $TP = \sum_i f_i \hat{f}_i$, $FP = \sum_i (1 - f_i) \hat{f}_i$, $FN = \sum_i f_i (1 - \hat{f}_i)$, and $TN = \sum_i (1 - f_i) (1 - \hat{f}_i)$. False-pass precision, recall, and F1 use their standard binary definitions. Because an abstention is neither a repair prediction nor an acceptance, False-Acceptance Rate (FAR) is reported separately:

$$\text{FAR} = \frac{\sum_i f_i \mathbb{I}(\hat{y}_i = \text{VERIFIED})}{\sum_i f_i}. \quad (12)$$

Here, y_i and $\hat{y}_i$ are the gold and predicted decisions in $\{\text{VERIFIED}, \text{REPAIR_REQUIRED}, \text{ABSTAIN}\}$. Decision accuracy counts an exact decision-label match over all 73 cases, so a predicted ABSTAIN is correct for this metric when the gold decision is also ABSTAIN. With $c_i = \mathbb{I}(\hat{y}_i \neq \text{ABSTAIN})$,

decision coverage and selective accuracy are

$$\begin{aligned} \text{Coverage} &= \frac{\sum_i c_i}{N}, \\ \text{Selective accuracy} &= \frac{\sum_i c_i \mathbb{I}(\hat{y}_i = y_i)}{\sum_i c_i}. \end{aligned} \quad (13)$$

Attribution Macro-F1 is the unweighted mean of cause-level F1 scores over the seven gold defect classes `ASSERTION_ERROR`, `COMPILATION_ERROR`, `PLAN_TEST_MISMATCH`, `SETUP_OR_RUNTIME_ERROR`, `SHARED_INFRASTRUCTURE_ERROR`, `TARGET_NOT_REACHED`, and `WEAK_ORACLE`. Per-class precision, recall, and F1 are set to zero when their respective denominator is zero. Routing accuracy is the proportion of the 54 `REPAIR_REQUIRED` cases for which the predicted route belongs to the gold case’s acceptable-route set; the route vocabulary is `LOCAL_TEST_REPAIR`, `REGENERATE_TEST`, `SHARED_INFRASTRUCTURE_REPAIR`, `REQUEST_TOOL_EVIDENCE`, and `NONE`. Thus, a case with multiple acceptable routes is correct when the prediction matches any listed route rather than only a canonical first route. The overall abstention rate uses all 73 cases. The Qwen run displayed below has no invalid case-level outputs.

Table 7 summarizes the safety and decision-quality outcomes.

Table 8 exposes the confusion counts behind the false-passing metrics. The false-accepted and positive-abstention columns are subsets of *FN*; the remaining false negatives are cases assigned another non-positive repair cause.

Gemini produced consistently favorable point estimates for B2: it detected 16/26 false-passing cases instead of 11/26, had 7 rather than 15 false acceptances, and had a 9.73-percentage-point higher selective accuracy; in return, B2 generated 8 false positives and abstained on 13 cases. On the DeepSeek repaired set, B2 had 11 fewer false acceptances, but Decision Accuracy was 10.96 points lower alongside lower coverage. On Qwen3.8 Flash, B2 detected 20/26 rather than 11/26 false-passing cases and reduced false acceptances from 10 to 2. Its false-pass F1 increased by 17.46 points, while it produced 6 false positives and the paired decision-accuracy test did not show a clear difference.

4.5.2 Attribution, routing and uncertainty

Table 9 shows that B2’s observed routing accuracy was 24.08 percentage points higher on Gemini and 9.26 points higher on DeepSeek, but unchanged on Qwen3.8 Flash. Its observed attribution Macro-F1 was 2.35 and 0.09 points higher on Gemini and DeepSeek, respectively, but 23.65 points lower on Qwen. This substantial Qwen diagnosis loss limits any claim of overall B2 benefit despite its false-acceptance improvement. These differences are descriptive point estimates, not causal evidence specifically for role separation, because B2 simultaneously uses multiple specialist calls and a larger inference budget.

Table 10 reports the paired uncertainty analyses.

4.5.3 Empirical integrity and cost

Across the three B2 specialist calls for each of 73 cases, Gemini had 219/219 valid specialist outputs after the failed rows were rerun; this count excludes B1 and the deterministic aggregator. The final DeepSeek set also had 219/219 valid specialist outputs, but it is a repaired artifact: six rows were replaced, including four from a targeted rerun on 31/08 using model ID `deepseek-v4-pro` after the 26/08 freeze had already been scored. The repair raised B2’s False-pass F1 from 63.16% to 70.00% and reversed the Attribution difference from -2.06 to +0.09 points. Therefore, the repaired DeepSeek set is treated only as sensitivity evidence; a confirmatory experiment must rerun the entire cohort under a new retry/freeze protocol before the gold labels are opened. The Qwen run had 219/219 valid B2 specialist outputs and no invalid case-level outputs. B2 used 2.97 times as many input tokens as B1 on Gemini and DeepSeek and 2.07 times as many on Qwen; its output-token usage was 2.43, 1.24, and 1.32 times that of B1, respectively. Qwen B2 used 221 rather than 83 API calls. This cost trade-off must be considered together with FAR, coverage, abstention, and attribution.

Qwen was chosen for display after inspecting four additional complete System 2 runs on this same cohort. Their B2-minus-B1 false-pass F1 differences were +2.08 points for DeepSeek V4 Flash,

Table 7 Safety and quality outcome measures

Model/run	Verifier	False-pass F1	FAR	Decision accuracy	Coverage	Selective accuracy
Overall	B0	0.00%	100.00%	45.21%	100.00%	45.21%
Gemini 3.5	B1	59.46%	57.69%	60.27%	100.00%	60.27%
Gemini 3.5	B2	64.00%	26.92%	65.75%	82.19%	70.00%
DeepSeek (repaired)	B1	70.00%	42.31%	72.60%	87.67%	71.88%
DeepSeek (repaired)	B2	70.00%	0.00%	61.64%	76.71%	80.36%
Qwen3.8 Flash sensitivity	B1	59.46%	38.46%	73.97%	72.60%	81.13%
Qwen3.8 Flash sensitivity	B2	76.92%	7.69%	78.08%	82.19%	81.67%

Table 8 False-passing confusion counts and unresolved positive cases

Model/run	Verifier	TP	FP	FN	TN	False accepted	Positive abstentions
Overall	B0	0	0	26	47	26	0
Gemini 3.5	B1	11	0	15	47	15	0
Gemini 3.5	B2	16	8	10	39	7	3
DeepSeek repaired	B1	14	0	12	47	11	1
DeepSeek repaired	B2	14	0	12	47	0	9
Qwen3.8 Flash sensitivity	B1	11	0	15	47	10	5
Qwen3.8 Flash sensitivity	B2	20	6	6	41	2	2

Table 9 Diagnosis and repair routing on 54 REPAIR_REQUIRED cases; abstention uses all 73 cases

Model/run	Verifier	Attribution Macro-F1	Routing accuracy	Overall abstention
Overall	B0	7.35%	31.48%	0.00%
Gemini 3.5	B1 / B2	35.21% / 37.56%	42.59% / 66.67%	0.00% / 17.81%
DeepSeek repaired	B1 / B2	37.40% / 37.49%	53.70% / 62.96%	12.33% / 23.29%
Qwen3.8 Flash sensitivity	B1 / B2	45.49% / 21.84%	59.26% / 59.26%	27.40% / 17.81%

Table 10 Paired tests and uncertainty

Model/run	McNemar p Decision	Delta False-pass F1 [bootstrap 95% CI]	McNemar p FAR*	McNemar p Routing*
Gemini 3.5	0.481	+4.54 [-10.23; 22.22]	0.0078	0.0010
DeepSeek repaired	0.115	0.00 [-9.77; 9.62]	0.0010	0.2266
Qwen3.8 Flash sensitivity	0.664	+17.46 [0.00; 37.29]	0.0078	1.0000

* Supplementary exact McNemar tests, not adjusted for multiple testing. The cases are still clustered by repository and base test; therefore, they should be revalidated using a project-clustered bootstrap or an independent holdout.

+8.27 for Gemini 3.7 (Google), +10.01 for Gemini 3.7 (local medium), and +17.46 for Qwen3.8 Flash. This outcome-informed choice favors B2’s safety result and is not a prespecified model comparison. The other three runs remain available in the local results package; none established an across-metric improvement. The earlier GLM run is omitted from the numerical comparison because 87 of its 219 case-level outputs were invalid or partially invalid. The Qwen interval for the F1 difference includes zero at its lower bound, and the paired decision-accuracy test is not significant; no confirmatory superiority claim follows from these exploratory analyses.

4.5.4 Answering RQ3

On the v2 exploratory cohort, B2 had its largest observed routing difference on Gemini 3.5; Qwen showed a larger false-pass F1 increase and fewer false acceptances, but its attribution Macro-F1 fell by 23.65 points and routing did not improve. B2 uses a larger inference budget, DeepSeek relies on a repaired set, and Qwen was selected after inspecting alternatives. The results support further study of B2 as a safety-oriented verification layer, but do not generally confirm Contribution 3; an independent, project-aware holdout frozen before model selection and scoring is needed.

4.6 Exploratory End-to-End TRACE System Evaluation against EvoSuite

4.6.1 Comparison Design and Statistical Methods

In this comparison, TRACE is the integrated pipeline combining structural planning, traceable generation, tool execution, B2 verification, and at most two TGSLR rounds. We compare TRACE with EvoSuite 1.2.0 [8] on a frozen cohort of 150 focal targets from 58 SF110 projects, version SF110-20130704 [9]. Each target was run with three fixed repetitions, 42, 43, and 44. The design yields 450 TRACE–EvoSuite pairs on the same target and repetition. Both treatments were evaluated through a common Maven/Surefire, JaCoCo, and PIT evaluator and a treatment-blinded B2 gate. The available campaign description does not establish that this TRACE-oriented gate is

construct-neutral for EvoSuite. The written protocol specifies a 900-second hard wall-clock budget for each target–repetition–treatment and a 420-second EvoSuite search allocation. However, four rows in the paired export report a 120-second EvoSuite process timeout. Historical launch arguments and configuration snapshots are needed to reconcile these limits; the protocol alone does not establish equal realized budgets. TRACE had at most two TGSLR rounds, whereas EvoSuite had no repair round.

The primary analysis follows the delivered-system intention-to-treat (ITT) principle. A candidate that was not delivered or was rejected by the verifier received a value of 0 for the delivered outcomes. MissingEvidence was neither assigned 0 nor removed by complete-case deletion. The campaign completed all 450/450 pairs, corresponding to 900 terminal records, with no MissingEvidence. Therefore, the entire cohort was retained in the analysis.

The unit of inference is the target. Before testing, the three repetitions for each target were averaged. The reported 95% confidence intervals use 2,000 hierarchical cluster-bootstrap replicates in project, target, and repetition order. The reported p values use a two-sided paired sign-flip permutation test with 10,000 permutations on the 150 target-level differences. Holm–Bonferroni correction was applied to the protocol’s family comprising delivered Compilation Success Rate (delivered CSR), delivered Execution Success Rate (delivered ESR), STS, branch coverage, and mutation score. Verified-Suite Rate (VSR), the proportion of pairs ending in a B2-verified delivered suite, is the primary outcome and is reported with an unadjusted permutation p value. A fixed seed of 20260919 was used for sampling. The hierarchical confidence intervals account for project resampling, but this does not make target-level sign flips project-clustered tests; the latter still require an exchangeability assumption for target differences. The paired export does not report raw pre-gate CSR or ESR.

4.6.2 Delivered-System ITT Results

Table 11 reports the ITT outcomes for the two treatments.

Table 11 ITT results for TRACE and EvoSuite across 450 paired observations

Metric	TRACE	EvoSuite	Delta	95% CI for Delta	p	p_{Holm}
VSR	55.11%	28.89%	+26.22	[14.22; 38.36]	0.0001	–
Delivered CSR	55.11%	28.89%	+26.22	[14.22; 38.36]	0.0001	0.0005
Delivered ESR	55.11%	28.89%	+26.22	[14.22; 38.36]	0.0001	0.0005
STS	47.34%	28.89%	+18.45	[8.01; 29.91]	0.0002	0.0006
Branch coverage	42.23%	28.32%	+13.91	[3.20; 25.69]	0.0007	0.0007
Mutation score	44.03%	26.89%	+17.14	[6.83; 28.23]	0.0002	0.0006

Delta = TRACE minus EvoSuite, in percentage points. CI denotes the percentile 95% confidence interval from the hierarchical cluster bootstrap. The p values are from the two-sided paired permutation test. VSR was not included in the prespecified Holm family.

TRACE achieved a VSR of 55.11% (248/450), higher than EvoSuite’s 28.89% (130/450), a difference of 26.22 percentage points. The 95% confidence interval [14.22; 38.36] does not include 0, and the permutation test gave $p = 0.0001$. Delivered CSR and delivered ESR had the same difference, with $p_{\text{Holm}} = 0.0005$. Within the evaluated cohort and TRACE-specific evaluation contract, these results show a higher probability that TRACE produced a suite that was delivered and then counted as compiled and executed after the verification gate; they do not compare raw pre-gate compilation or execution rates.

All three adequacy metrics were also statistically significantly higher. STS increased by 18.45 percentage points, with a 95% CI of [8.01; 29.91] and $p_{\text{Holm}} = 0.0006$. Branch coverage increased by 13.91 percentage points, with a 95% CI of [3.20; 25.69] and $p_{\text{Holm}} = 0.0007$. Mutation score increased by 17.14 percentage points, with a 95% CI of [6.83; 28.23] and $p_{\text{Holm}} = 0.0006$. All confidence intervals exclude 0 when all terminal outcomes are retained under ITT. These results support higher delivered-system test adequacy within the frozen cohort, budget, and tool configuration.

4.6.3 Target-Level Paired Analysis

Among the 450 repetition-level pairs, only TRACE had a verified suite in 148 pairs, only EvoSuite had a verified suite in 30 pairs, both had a verified suite in 100 pairs, and neither had a verified suite in 172 pairs. After averaging the three repetitions for each target, the numbers of targets where TRACE won, EvoSuite won, and the treatments tied were 62/14/74 for VSR, 59/27/64 for branch coverage, and 60/19/71 for mutation score, respectively. The statistical analysis used

these 150 target-level differences to avoid treating the three repetitions of the same target as independent observations.

4.6.4 Candidate Diagnostics before the Verification Gate

Table 12 reports candidate metrics before the delivered-system gate.

TRACE had a higher candidate point estimate on all three metrics. The gap between candidate and delivered outcomes shows that some outputs had quality measurable by the tools but did not pass all generation, repair, and verification gates. Therefore, Table 12 is used to explain pipeline behavior, while the main conclusion rests on Table 11.

4.6.5 Outcome Integrity and Cost

TRACE had 248 runs verified by B2. The remaining terminal outcomes comprised 75 budget-exhausted, 52 repair-exhausted, 41 B2-abstention, 17 verifier-rejected, 12 runs in the export’s tool-failure category, and 5 provider-failure runs. The 12 tool-failure rows describe ten failures to construct TGSLR context from the finalized plan and two zero-scenario plans; this taxonomy should not be interpreted as twelve independent JaCoCo/PIT measurement crashes. EvoSuite had 130 verified runs. Of its 175 generation-failure rows, 171 report no generated test file within budget and four report a 120-second process timeout. Another 139 rows have the explicit reason `TerminalRepairExhausted`, `NoRepairInBaseline`, and six end in unverified abstention. The shared repair-exhausted label therefore does not imply an EvoSuite repair round. These terminal states were retained in the ITT

Table 12 Candidate metrics before applying the delivered-system gate

Metric	TRACE	EvoSuite	Delta	95% CI for Delta
Branch coverage	70.84%	46.25%	+24.59	[13.54; 36.12]
Mutation score	61.76%	41.12%	+20.64	[10.15; 31.42]
STS	75.97%	45.79%	+30.18	[19.77; 41.16]

Candidate diagnostics describe Maven, JaCoCo, and PIT evidence before the delivered-system gate. The confidence intervals are diagnostic estimates; they do not replace the ITT outcomes and are not part of the primary Holm testing family.

sample; their underlying generator-versus-harness causes require the corresponding execution logs.

The recorded total duration was 19.65 hours for TRACE and 14.73 hours for EvoSuite, corresponding to means of 157.18 and 117.81 seconds per target-repetition for each treatment, respectively. TRACE used 37,564,632 recorded provider tokens, averaging 83,477 per run. The EvoSuite token column is zero throughout the export, but the available accounting does not establish whether shared B2-verification calls are included for that arm. Zero in this column is therefore not evidence of zero total inference cost for the evaluated EvoSuite pipeline. The duration totals are sums of per-treatment run durations, not elapsed campaign time, and do not describe parallelism.

4.6.6 Interpretation and Limits of the Conclusion

TRACE achieved higher delivered-system outcomes than EvoSuite with statistical significance on the reported ITT outcomes in the recorded campaign. This conclusion applies to the integrated pipeline under the fixed SF110 cohort, tool lock, budget, and TRACE-specific verifier. Treatment blinding does not by itself establish that the B2 gate is construct-neutral across the two generators. The comparison also does not isolate the causal effects of Structural Planning, TGSLR, or B2; those components must be interpreted through RQ1, RQ2, and RQ3, respectively. The results do not establish TRACE’s superiority on every benchmark, LLM, budget, tool version, or evaluation contract.

The campaign identifier is:

official-150-20260920-r42-r44-v1

Recalculation from the local 450-row paired-comparison export using the workspace’s bootstrap and permutation implementations

reproduces the point estimates, confidence intervals, and reported significance levels in Tables 11 and 12 to their displayed precision. This checks numerical consistency, not historical execution. The cohort must be distinguished from a separate 45-pair export labeled **phase11-official-export**. Verification scripts and recalculated reports are retained with the manuscript; the empirical inputs and original statistical engines remain in the wider workspace. Historical launch configuration and raw-evidence linkage still require a versioned package connecting the 450-pair CSV, cohort manifest, tool lock, redacted launch configuration, analysis code, seeds, and raw evidence through a hash manifest.

5 Threats to Validity

5.1 Construct validity

Compilation, execution, structural-target satisfaction, coverage, mutation score, and verifier decisions capture different properties and are not interchangeable proxies for test correctness. RQ2 records terminal compilation-stage evidence and whole-run duration, but neither independently establishes scenario conformance or oracle strength. Missing adequacy measurements restrict its coverage and mutation results to descriptive complete-case summaries. In RQ3, the false-passing definition combines four semantic failure causes, while abstention is treated separately from both repair prediction and acceptance; Equations 12 and 13 make that distinction explicit. For the TRACE comparison, delivered CSR and delivered ESR are post-gate outcomes, not raw generation rates, and treatment blinding does not establish construct-neutrality of the TRACE-specific B2 gate for EvoSuite.

5.2 Internal validity

Most component configurations were run once per experimental unit, so provider variability, retry behavior, and uncontrolled nondeterminism may affect the estimates. RQ2 combines an initial 99-unit campaign and a later replacement unit under provisional plans; its 100-unit reanalysis does not establish prospective preregistration. For RQ3, the author team reports oral review of the final gold labels but kept no case-level adjudication record; the 73 cases came from an earlier 180-case candidate pool exposed to a prototype, and there was no independently labeled holdout. The repaired DeepSeek artifact was modified after the original freeze and is therefore sensitivity evidence only. Qwen was chosen after viewing four complete alternative runs, introducing outcome-based selection bias. The current studies also do not isolate the effect of specialist role separation from B2’s larger inference budget, nor do they isolate individual TRACE components in the TRACE comparison. For that comparison, historical timeout settings and shared-verifier cost accounting remain unresolved.

5.3 External validity

The evaluated models, Java projects, SF110 targets, LIBRO–Defects4J error units, and RQ3 cases cover only a limited sample of repositories, faults, and provider configurations. Results may not generalize to other languages, build systems, test frameworks, model snapshots, budgets, or verifier contracts. The TRACE–EvoSuite findings are consequently restricted to the recorded cohort and TRACE-specific evaluation contract.

5.4 Conclusion validity and reproducibility

The data are clustered by project, while not all analyses fully model project clustering or between-run variability. In particular, the integrated comparison’s target-level permutation tests are distinct from its hierarchical confidence intervals. Multiple comparisons were Holm-adjusted where specified, but correction does not turn an exploratory reanalysis into a confirmatory design or repair selection bias. The exploratory

RQ2/RQ3 tests and complete-case adequacy summaries do not support broad causal claims. Reproduction depends on exact prompts, configurations, model and tool versions, seeds, target manifests, raw records, and analysis scripts. Local CSV and metric reports permit numerical consistency checks, but do not by themselves establish raw-evidence provenance or independent experimental replication. These artifacts must be linked to the reported campaigns in a versioned, accessible package before reproducibility can be independently audited.

6 Conclusion and Future Work

This study proposes TRACE, a framework for generating, verifying, and repairing Java unit tests based on LLMs, designed to address three main limitations of existing methods: the lack of an auditable test planning step before code generation, the lack of a stable traceability mechanism to control the scope of repair, and the lack of a specialized verification layer capable of linking tool evidence to each test scenario. TRACE implements three corresponding contributions: Auditable Test-Design-Technique-Grounded Structural Planning, Bijective Scenario Ownership and Regression-Checked Scenario-Local Repair, and Tool-Grounded Specialized Multi-Agent Verification for Scenario-Level Failure Attribution. Instead of viewing the test case as a single LLM call, the framework maintains a trace chain from the source code structure, atomic scenario, generated test method, execution evidence to the verification decision and repair route.

For RQ1, 600 runs on 150 SF110 focal methods across two LLM configurations associate the PLANNED package with higher end-to-end reliability. Aggregate execution success increased from 58.00% to 88.33%. The descriptive branch-coverage mean increased from 43.51% to 70.58%, but the DeepSeek non-N/A denominators differ by treatment and lack reason codes. Complete-pair improvements in execution success, structural-target satisfaction, branch coverage, and mutation score reached significance after Holm correction in both configurations; compilation success did so only on DeepSeek. Conditional adequacy benefits were clearer on Gemini and less stable on DeepSeek. Planning consumed approximately 2.03 times as many tokens and increased wall-clock

time, so the comparison does not isolate plan representation from additional inference compute.

For RQ2, an exploratory reanalysis of 100 defect units found core repair success of 60% for TGSLR, 46% for FTMR, and 76% for FCR. TGSLR’s advantage over FTMR did not reach significance after Holm correction; the reported tests favored FCR for success and token/API/iteration cost. On their respective successful subsets, TGSLR patches changed 18.45 lines on average, 10.48 fewer than FCR, but this comparison is descriptive. Regression occurred in 6/100 TGSLR runs and 8/100 FCR runs, with no significant paired difference. Missing coverage and mutation evidence prevents a conclusion of adequacy non-inferiority. These results support neither overall TGSLR superiority nor a claim that traceability itself improved measured effectiveness; the locality trade-off requires confirmation under a prospectively frozen analysis plan.

For RQ3, an exploratory evaluation on 73 cases across 14 Java projects suggests that the Specialized Multi-Agent Verifier can reduce false acceptance under some model configurations. On Gemini 3.5, B2 had a lower observed false-acceptance rate than the General-Purpose Reviewer (26.92% versus 57.69%) and a higher observed routing accuracy on cases needing fixes (66.67% versus 42.59%). Its selective accuracy was also higher (70.00% versus 60.27%), while its decision coverage was lower (82.19% versus 100%). On Qwen3.8 Flash, B2 reduced false acceptances from 10 to 2 but attribution Macro-F1 fell from 45.49% to 21.84% and routing remained at 59.26%. B2 used 2.97 times more input tokens than B1 on these runs. DeepSeek relies on repaired artifacts, and Qwen was selected after inspecting alternatives. The author team reports oral review of the final gold labels, with no written case-level adjudication record. These results do not generally confirm Contribution 3 or attribute the observed differences causally to role specialization.

In the separate integrated evaluation, TRACE and EvoSuite were compared on 450 target-repetition pairs from 150 SF110 targets. TRACE achieved a VSR of 55.11%, compared with 28.89% for EvoSuite; the 26.22-percentage-point difference was statistically significant in the reported paired permutation test ($p = 0.0001$). The delivered-system ITT adequacy metrics also favored TRACE after Holm correction. Recorded

treatment durations totaled 19.65 hours for TRACE and 14.73 hours for EvoSuite, while TRACE used 37,564,632 recorded provider tokens. The zero-valued EvoSuite token column does not establish whether shared B2 costs were counted. This result applies to the recorded cohort and TRACE-specific evaluation contract; historical timeout reconciliation, complete cost accounting, project-level testing sensitivity, and gate neutrality remain unresolved. It does not isolate any one TRACE component or establish superiority in other settings.

Overall, the results indicate that the quality of unit tests generated by LLMs does not solely depend on the model’s code-generation capability. The reliability of the entire process also depends on how the testing objectives are represented before generation, how each test is linked to the corresponding scenario, how the scope of edits is controlled, and how evidence from compilers, JUnit, coverage, and mutation testing is synthesized. Among the component studies, Structural Planning provides the clearest empirical evidence of improving end-to-end reliability; Scenario-Local Repair primarily offers control and traceability rather than an absolute advantage in success rate; and Specialized Multi-Agent Verification demonstrates potential value in contexts where false acceptance carries a high cost.

Section 5 details the construct, internal, external, conclusion, and reliability threats. In particular, the results must be interpreted within the evaluated datasets, models, tool configurations, budgets, and evidence contracts rather than as evidence of the universal superiority of TRACE.

In the future, studies need to be repeated on multiple LLMs, seeds, repositories, and defect datasets, with multiple independent runs for each configuration. Evaluation should use project-aware or hierarchical statistical analysis, while also constructing a holdout for RQ3 that is frozen before running the verifier and labeled by multiple independent human annotators. Coverage and mutation evidence also need to be consistently collected at the test method level through test-coverage matrices and test-mutant matrices. Additionally, adaptive repair strategies can be investigated, in which the system prioritizes scenario-local repair but automatically switches to shared-infrastructure repair or full-class regeneration when evidence shows the defect exceeds

the scope of a single scenario. Another important direction is to reduce the cost of specialized verifiers, fine-tune the abstention policy, and determine the conditions under which the safety benefits are large enough to compensate for the additional tokens and time. With these improvements, TRACE can move towards a process of generating unit tests using LLMs that is both efficient and more auditable, traceable, and reliably verifiable.

Acknowledgements. To be completed by the authors.

Statements and Declarations

Funding

To be completed by the authors.

Competing interests

To be completed by the authors.

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

To be completed by the authors.

Code availability

To be completed by the authors.

Author contributions

To be completed by the authors.

References

- [1] Ahammad A, Bajta ME, Radgui M (2025) MAGISTER: Llm-based test generation with role-specialized agents. In: 2025 International Conference on Intelligent Systems: Theories and Applications, pp 1–7, <https://doi.org/10.1109/SITA67914.2025.11273637>
- [2] Almasi M, Hemmati H, Fraser G, et al (2017) An industrial evaluation of unit test generation: Finding real faults in a financial application. In: 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track, <https://doi.org/10.1109/ICSE-SEIP.2017.27>
- [3] Alshahwan N, Chheda J, Finogenova A, et al (2024) Automated unit test improvement using large language models at meta. In: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, <https://doi.org/10.1145/3663529.3663839>
- [4] Chang P, Fang Y, Chen S, et al (2026) Test vs mutant: Adversarial llm agents for robust unit test generation. In: International Symposium on Software Testing and Analysis (ISSTA 2026), Research Papers, accepted research paper; proceedings DOI pending
- [5] Chen J, Wang Z, Yang L, et al (2026) Context matters: Improving the practical reliability of llm-based unit test generation. In: International Symposium on Software Testing and Analysis (ISSTA 2026), Research Papers, accepted experience paper; proceedings DOI pending
- [6] Chen Y, Hu Z, Zhi C, et al (2024) ChatUnitTest: A framework for llm-based test generation. In: Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, Demonstrations Track, [arXiv:2305.04764](https://arxiv.org/abs/2305.04764)

- [7] Foster C, Gulati A, Harman M, et al (2025) Mutation-guided llm-based test generation at meta. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp 180–191, <https://doi.org/10.1145/3696630.3728544>
- [8] Fraser G, Arcuri A (2011) EvoSuite: Automatic test suite generation for object-oriented software. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, pp 416–419, <https://doi.org/10.1145/2025113.2025179>
- [9] Fraser G, Arcuri A (2014) A large-scale evaluation of automated unit test generation using EvoSuite. *ACM Transactions on Software Engineering and Methodology* 24(2):1–42. <https://doi.org/10.1145/2685612>
- [10] Ghosh T, Abbas N (2025) Retry and refine: A multi-agent framework for llm-based reliable unit test generation. In: Artificial Intelligence XLII, Lecture Notes in Computer Science, vol 16301. Springer Nature Switzerland, Cham, pp 92–105, https://doi.org/10.1007/978-3-032-11402-0_7
- [11] Gu S, Zhang Q, Li K, et al (2026) TestART: Improving llm-based unit testing via co-evolution of automated generation and repair iteration. *ACM Transactions on Software Engineering and Methodology* <https://doi.org/10.1145/3844945>
- [12] Hong S, Zhuge M, Chen J, et al (2024) MetaGPT: Meta programming for a multi-agent collaborative framework. In: International Conference on Learning Representations, [arXiv:2308.00352](https://arxiv.org/abs/2308.00352)
- [13] Hossain SB, Dwyer MB (2025) TOGLL: Correct and strong test oracle generation with llms. In: 2025 IEEE/ACM 47th International Conference on Software Engineering, pp 1475–1487, <https://doi.org/10.1109/ICSE55347.2025.00098>
- [14] ISO/IEC/IEEE (2021) ISO/IEC/IEEE 29119-4:2021: Software and systems engineering—software testing—part 4: Test techniques. International Standard, URL <https://www.iso.org/standard/79430.html>
- [15] JaCoCo Project (2026) JaCoCo java code coverage library: Documentation. Project documentation, URL <https://www.jacoco.org/jacoco/trunk/doc/>
- [16] Jin X, Liu Z, Zhang J, et al (2025) Tracets4J: A traceable unit test generation dataset. In: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering, pp 757–767, <https://doi.org/10.1109/SANER64311.2025.00077>
- [17] JUnit Team (2026) JUnit 5 user guide. Project documentation, URL <https://junit.org/junit5/docs/current/user-guide/>
- [18] Just R, Jalali D, Ernst MD (2014) Defects4J: A database of existing faults to enable controlled testing studies for java programs. In: Proceedings of the 2014 International Symposium on Software Testing and Analysis, pp 437–440, <https://doi.org/10.1145/2610384.2628055>
- [19] Kang S, Yoon J, Yoo S (2023) Large language models are few-shot testers: Exploring llm-based general bug reproduction. In: 2023 IEEE/ACM 45th International Conference on Software Engineering, pp 2312–2323, <https://doi.org/10.1109/ICSE48619.2023.00194>
- [20] Nan Z, Guo Z, Liu K, et al (2025) Test intention guided llm-based unit test generation. In: 2025 IEEE/ACM 47th International Conference on Software Engineering, pp 1026–1038, <https://doi.org/10.1109/ICSE55347.2025.00243>
- [21] Pacheco C, Ernst MD (2007) Randoop: Feedback-directed random testing for java. In: Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications, pp 815–816, <https://doi.org/10.1145/1297846.1297902>

- [22] Peruma A, Almalki K, Newman CD, et al (2020) tsDetect: An open source test smells detection tool. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp 1650–1654, <https://doi.org/10.1145/3368089.3417921>
- [23] PIT Project (2026) PIT mutation testing: Documentation. Project documentation, URL <https://pitest.org/>
- [24] Pizzorno JA, Berger ED (2025) CoverUp: Effective high coverage test generation for python. Proceedings of the ACM on Software Engineering 2. <https://doi.org/10.1145/3729398>
- [25] Ryan G, Jain S, Shang M, et al (2024) Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. Proceedings of the ACM on Software Engineering 1(FSE):951–971. <https://doi.org/10.1145/3643769>
- [26] Schäfer M, Nadi S, Eghbali A, et al (2024) An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering 50(1):85–105. <https://doi.org/10.1109/TSE.2023.3334955>
- [27] Wang G, Xu Q, Briand L, et al (2026) Mutation-guided unit test generation with a large language model. IEEE Transactions on Software Engineering <https://doi.org/10.1109/TSE.2026.3682975>
- [28] Wang Z, Liu K, Li G, et al (2024) HITS: High-coverage llm-based unit test generation via method slicing. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp 1258–1268, <https://doi.org/10.1145/3691620.3695501>
- [29] Xu Q, Wang G, Briand L, et al (2026) Hallucination to consensus: Multi-agent llms for end-to-end junit test generation. ACM Transactions on Software Engineering and Methodology <https://doi.org/10.1145/3803418>
- [30] Yang J, Jimenez CE, Wettig A, et al (2024) SWE-agent: Agent-computer interfaces enable automated software engineering. In: Advances in Neural Information Processing Systems, [arXiv:2405.15793](https://arxiv.org/abs/2405.15793)
- [31] Yang R, Xu X, Wang R (2025) TestLoter: A logic-driven framework for automated unit test generation and error repair using large language models. Journal of Computer Languages 84:101348. <https://doi.org/10.1016/j.cola.2025.101348>
- [32] Yuan Z, Liu M, Ding S, et al (2024) Evaluating and improving chatgpt for unit test generation. Proceedings of the ACM on Software Engineering 1(FSE):1703–1726. <https://doi.org/10.1145/3660783>
- [33] Zhang Q, Gao C, Han Y, et al (2026) SGA-agent: Suggestion-guided llm-based multi-agent framework for repository-level software repair. ACM Transactions on Software Engineering and Methodology <https://doi.org/10.1145/3818617>